Preface

This is a book about deciding.

Not about AI in general, though you'll learn a lot of that on the way. It's about one narrow, very common job: looking at a situation and choosing what to do. Is this email phishing? Which team gets this ticket? Is this payment safe? Should this agent call that tool?

Software makes millions of these decisions every day, and more and more of them are handed to large language models. LLMs are superb at reading and writing. But a decision isn't a paragraph. It's a choice between a few options, made without being sure, with a price for getting it wrong. To make one well you need three things:

- a **probability you can trust**, one that means what it says;
- a **threshold set by what mistakes cost**, not by habit;
- a clear rule for **when to hand the case to a person**.

Most people building with AI today have none of the three. This book gives you all of them, from first principles, in plain language.

In 2026 a company called TypeSafe AI released Jev, which it calls the first **System One model**: a model that never writes, and only answers typed questions with probabilities, in one fast pass (TypeSafe AI 2026a). You'll learn to test whether Jev does what its makers claim. But the idea behind it outlives any one product: some decisions belong in a separate, fast, measurable layer of an AI system, and that layer needs different skills from the rest.

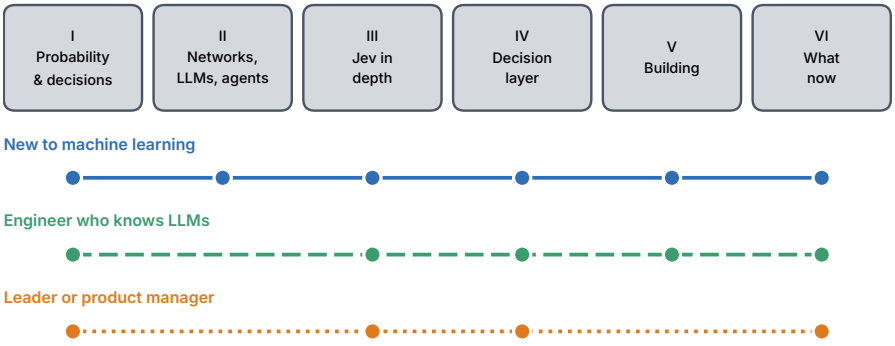
Three rules I followed

No invented measurements. I had no access to Jev's API while writing, so every Jev number comes from a stand-in that copies its public interface, and is marked *synthetic* every time. These numbers show you how to reason and test; they're not claims about the real model.

Vendor claims are labelled. When I report what TypeSafe says about Jev, I say it's *vendor-reported* and cite the source.

Claims go in piles. Everything said about Jev is sorted into what I could check, what the vendor reports, and what nobody outside the company knows yet. Chapter 9 explains how. Use the same three piles for anything you read about new models, this book included.

How to read this book



Part I is the foundation for every route: probabilities, calibration and costs. Part II is optional if you know it.

Figure 1. Three routes through the book's six parts.

New to machine learning? Read it in order. Part I is the foundation: what a probability means and how to turn one into an action. Part II is a short tour of neural networks, LLMs and agents for people who haven't met them.

Already build with LLMs? Read Part I anyway, especially Chapter 3 (calibration) and Chapter 4 (costs). Everything after depends on them. Then go to Part III.

Lead a team or a product? Read Part I, then Chapters 8 to 14, then 18 and 22. You'll be able to ask your team the four questions that matter: What does a mistake cost? Where's the threshold? Can we trust the probabilities? Who checks the warnings?

The boxes

A **Key idea** box, with a thick bar on its left, holds one sentence worth remembering a year from now. Each chapter ends with **Keep these**, the chapter's key ideas in one list.

Where this breaks shows the limits of what the chapter just taught. Read these most of all: they're the difference between knowing a method and knowing when to trust it. **Set the threshold** asks you to make a small decision with real costs; the answer follows in brackets. **Going deeper** holds optional maths that nothing later depends on.

The numbers

The book's running example is a made-up shipping company, Kestrel Logistics, and its security team. Its alerts come from a program that knows the true answer for every one, so we can measure exactly how many attacks a policy misses. That also means every number is an *example of a method*, not a fact about the world. Charts marked **synthetic** are made this way: trust the pattern, and test the numbers on your own data.

Every number about the Jev model comes from a stand-in that copies Jev's public interface. Those are marked *synthetic* too. What TypeSafe AI says about Jev is marked *vendor-reported*.

The code behind every figure is in the book's repository, github.com/Mukkandi-Sridhar/decide-dont-generate, for readers who want it. You don't need it to follow the book.

Prologue: two nights

Picture a shipping company's security team. Its computers raise a warning, called an **alert**, whenever something looks suspicious: a sign-in from a strange country, an email with an odd link, a laptop talking to an unknown server. Someone has to look at each alert and decide: real attack, or nothing?

Tonight one person is on duty. Checking an alert properly takes about twelve minutes, so they can check about five an hour.

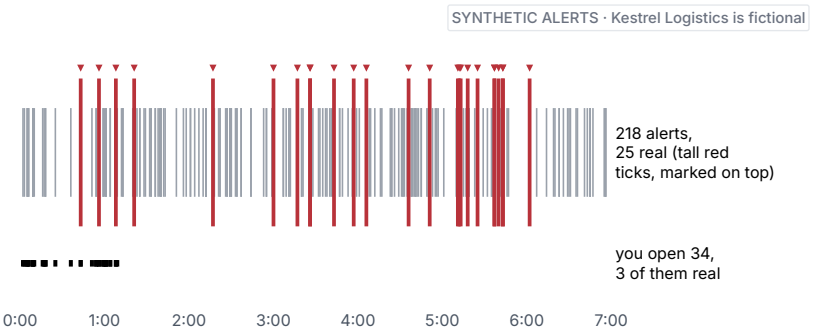


Figure 2. One night, midnight to 7 a.m. Each line is an alert; the real attacks are red, taller and marked on top. Working in the order they arrive, the person on duty checks the ones on the left. The rest wait for the morning.

Figure 2 shows the whole night. 218 alerts arrived between midnight and seven. 25 of them were real attacks. Working through them in the order they arrived, the person on duty checked 34, and only 3 of those were real.

They made no mistakes. Every alert they checked, they judged correctly. The problem was the order. The queue decided what they looked at, and a queue only knows what arrived first.

Now picture a second night.

A model reads every alert the moment it arrives. It writes nothing. It answers one question with a number: *how likely is this to be real?* And that number can be trusted: of the alerts it calls "one in ten", about one in ten really are attacks.

Alerts it is almost sure are harmless get closed, and a random few of those are kept aside to be checked later. Alerts it is fairly sure are dangerous wake someone up at once. Everything in between goes to the person on duty, most likely first.

The person still makes the decisions that matter. They just make them on the right alerts.

That second night is what this book builds. It needs surprisingly little new technology and quite a lot of clear thinking:

- what a probability really is, and how to tell if one can be trusted;
- what each kind of mistake costs, and where that puts the line between "act" and "ignore";
- when a machine should decide, and when it should hand the case to a person.

None of this is only about security. The same thinking runs a bank's fraud check, a hospital's waiting list and every AI agent that has to choose its next step. The security team is just the example we'll keep coming back to, because its mistakes are easy to count.

Next: before a machine can decide anything, it has to learn. What does that actually mean?

PART I

Prerequisites: probability and decisions

Before any neural network, any LLM or any Jev, every AI system has to answer one question well: *how sure are you?* These four chapters build the answer from the beginning. What learning is. What a probability means, and how a machine learns one. How to check whether it's calibrated. And how to turn it into an action, once you know what each mistake costs.

Everything later in the book is built on these pages. If you read only one part slowly, make it this one.

CHAPTER 1

What “learning” means

The question of whether a computer can think is no more interesting than the question of whether a submarine can swim.

— Edsger W. Dijkstra, 1984

AI. Machine learning. Deep learning. LLMs. RAG. Agents. If you’re getting into AI, these words are everywhere, and it can feel like seven separate subjects to learn.

They’re not. They’re chapters of one story, and each exists because the one before it hit a problem it couldn’t solve. This chapter tells that story from the start, and ends with where this book fits in it.

BY THE END OF THIS CHAPTER

- Tell the difference between a computer following rules and a computer learning.
- Say what a *model* is, in one plain sentence.
- See why a machine can learn exactly what you asked and still be useless.
- Place AI, machine learning, deep learning, LLMs, RAG, agents and Jev on one map.

Start with a rule

Inside a thermostat is a line that says, more or less: *if the temperature is above 30°C, turn on the fan.*

That’s a computer following a rule. Someone told it exactly what to do. It works nothing out.

A lot of software called “AI” is no more than this. A bank blocks payments over a limit. A spam filter deletes any email containing “lottery”. And rules are *good*: fast, cheap, and when one goes wrong you can read why. When a rule does the job, use it. In Chapter 13, plain rules beat far more complex models in more than one round.

The trouble starts when the world changes faster than you can write rules.

When rules run out

A company’s security team wants to catch *phishing*: emails that trick staff into typing their password into a fake page. Someone writes the obvious rule: *if an email mentions “password”, flag it.*

On Monday it flags the IT team’s own password-reset emails, so they add an exception. On Wednesday an attacker writes “password” with a zero. Another rule. On Friday the attacker drops the word completely and sends a link to a fake shared document.

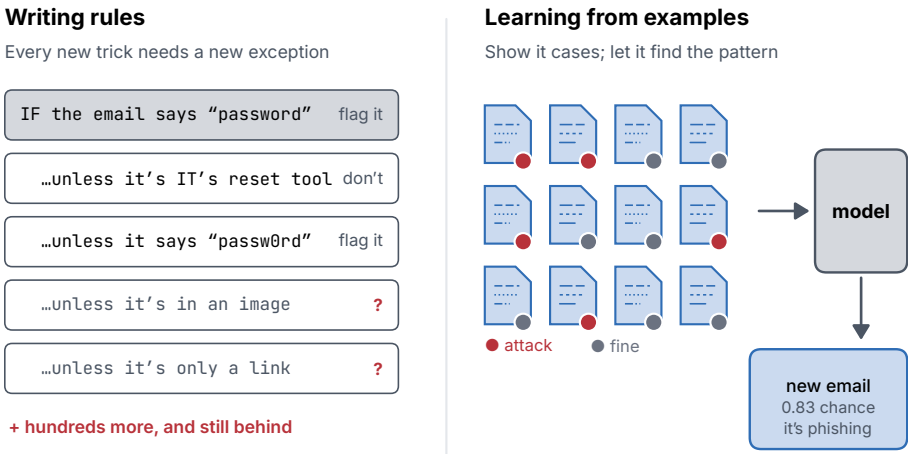


Figure 1.1. Two ways to catch phishing. On the left, every new trick forces a new exception, and the list never catches up. On the right, we stop writing rules and show the computer labelled examples instead.

The list of rules grows, and it’s always one trick behind (Figure 1.1). The attacker can learn your rules by testing them. You can’t read theirs.

Let the examples write the rules

So turn the job around. Give the computer thousands of emails where the answer is already known, *this one was phishing, this one was fine*, and let it find the pattern that separates them.

● KEY IDEA

Machine learning is getting a computer to work out the rules itself, from examples where the answer is already known.

It has been deciding things about you for years. When your bank texts *"Did you just spend ₹40,000 at an electronics shop in another city?"*, nobody wrote a rule about you. A system learned what your normal spending looks like, and this payment didn't fit.

So what's a "model"?

When the learning is done, it leaves something behind: a thing that takes an input and gives an answer. Give it an email, it says "phishing" or "fine". That thing is the model.

Picture a machine with many knobs. Learning means turning the knobs until the machine gets the examples right as often as possible. Then you stop turning and use it on new cases, where nobody knows the answer yet.

● KEY IDEA

A model is a function whose settings came from examples rather than from a programmer.

A machine learns, and learns the wrong thing

Now a real example. Our security team belongs to a made-up shipping company, Kestrel Logistics. In four weeks it receives 20,000 alerts, about 714 a day. Only 7.6% are real attacks; the rest are false alarms. (The company and its alerts are invented by a program, so we know the true answer for every one. That lets us check every model's work exactly.)

Each alert comes with a *threat score* between 0 and 1: how dangerous outside security sources think the sender or website is. An obvious rule: *flag the alert if its score is at least some number t.* A number like that, which splits cases into two groups, is a **threshold**.

Which *t*? We don't guess. We let the machine try every value from 0 to 1, count the mistakes each one makes, and keep the one with the fewest. That's learning in its smallest form: *try settings, count mistakes, keep the best.*

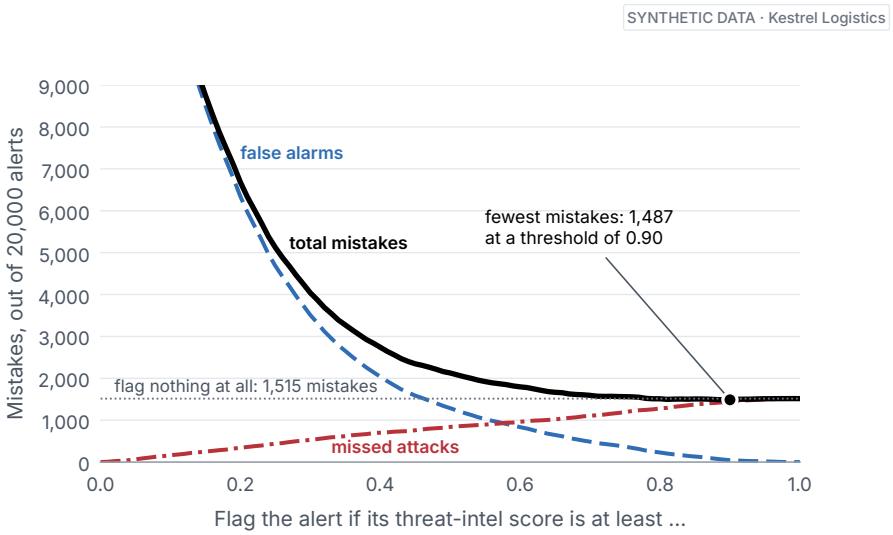


Figure 1.2. Every possible threshold and the mistakes it makes. Low thresholds bury the team in false alarms; high ones miss attacks. The total is lowest at the far right, just below the dotted line for “flag nothing at all”.

The machine picks 0.90: it flags only the very worst scores. It makes 1,487 mistakes. A “rule” that never flags anything makes 1,515, barely more. Of 1,515 real attacks, the winning rule catches 73.

Why? Harmless alerts outnumber attacks about twelve to one. So every false alarm hurts the mistake count more than catching an attack helps it, and the search pushes the threshold up until the rule almost stops raising alarms (Figure 1.2).

The machine did exactly what we asked. We asked the wrong question.

● KEY IDEA

A model learns exactly what you ask for: ask the wrong question, and it learns the wrong thing.

Two things went wrong. First, we counted every mistake as equal, but missing a real attack and wasting five minutes of someone’s time are not equal. Chapter 4 fixes that. Second, and this is what the book is really about, we forced the answer to be *yes* or *no*.

⚠ WHERE THIS BREAKS

A model only knows the world its examples came from. If attackers invent a new trick, the model has never seen it, and its answer will look just as confident as always. Labels can be wrong too: some attacks are never found, so they sit in the data marked "fine", and the model learns that mistake as if it were true.

The answer isn't yes or no

Ask a better question: *among alerts with a given score, how many turned out to be attacks?*

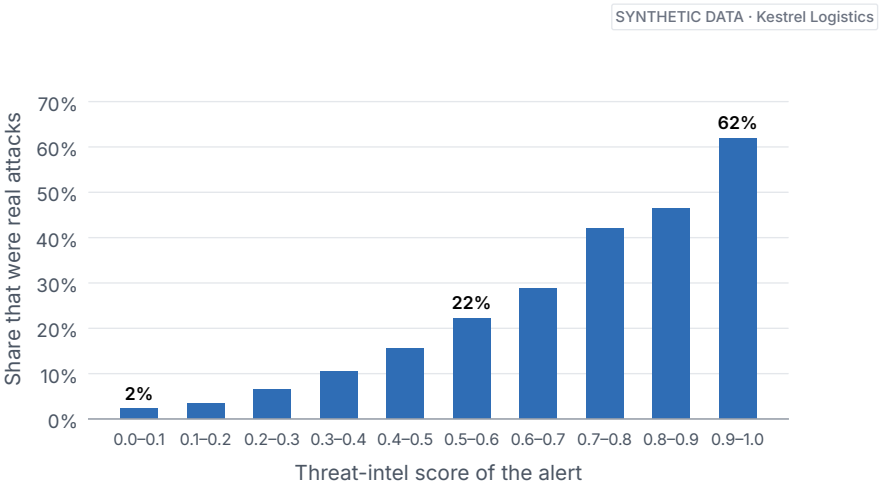


Figure 1.3. Group the alerts by threat score and count how many were real attacks. The share climbs steadily from 2% to 62%. Even at the top, plenty are harmless; even at the bottom, a few are attacks.

There's no point where alerts switch from harmless to dangerous (Figure 1.3). Among the lowest scores, about 2% were attacks. That sounds tiny, but it's 7,270 alerts, so over a hundred attacks hide among the most harmless-looking ones. Among the highest scores, 62% were attacks, so more than a third of the scariest-looking alerts were nothing.

So what do we know about an alert scoring 0.55? Not "attack". Not "fine". We know *about one in five alerts like this is an attack.*

That number tells the team how worried to be. It lets them check the likeliest attacks first. And once they know what each mistake costs, it tells them where to draw the line.

KEY IDEA

A good model doesn’t just answer: it says how sure it is, and it’s right about how sure it is.

Being *right about how sure it is* has a name: **calibration**. It’s a different goal from “make the fewest mistakes”, and Chapter 3 is all about it.

SET THE THRESHOLD

Missing a real attack costs the company about ten times what a false alarm costs. Should the threshold go *up* from 0.90 or *down*?
(Down, a lot. When a miss hurts more, you accept more false alarms to avoid one. Chapter 4 turns this into a formula.)

The rest of the story

A threat score is one handy number. But a photo is millions of pixels and an email is a string of words; nobody can hand-write the clues for “this sounds like a scam”. So the next step was models that learn their own clues, in many stacked layers: **deep learning** (Chapter 5).

Train a huge deep-learning model on a huge amount of text, with one task, *guess the next word*, and it picks up grammar, facts and some reasoning along the way. That’s a **large language model**, or LLM (Chapter 6).

An LLM knows a lot about the world and nothing about *your* world, so we let it look things up before it answers: **retrieval-augmented generation**, or RAG. And once a model can read, we want it to *act*: check a log, block an account, choose the next step. A model working in that loop is an **agent** (Chapter 7).

Where this book fits

Watch an agent work and you notice something. Most of its steps aren’t writing. They’re small decisions with a few possible answers. *Is this alert real? Which team gets this ticket? Is this action safe?*

Today, each of those usually goes to an LLM, which writes a paragraph, and then more code digs the answer out of the paragraph. It works. It’s also like hiring a novelist to tick a checkbox.

In September 2026 a start-up, TypeSafe AI, released **Jev**, a model built only for those checkbox moments. It never writes text. You give it a situation

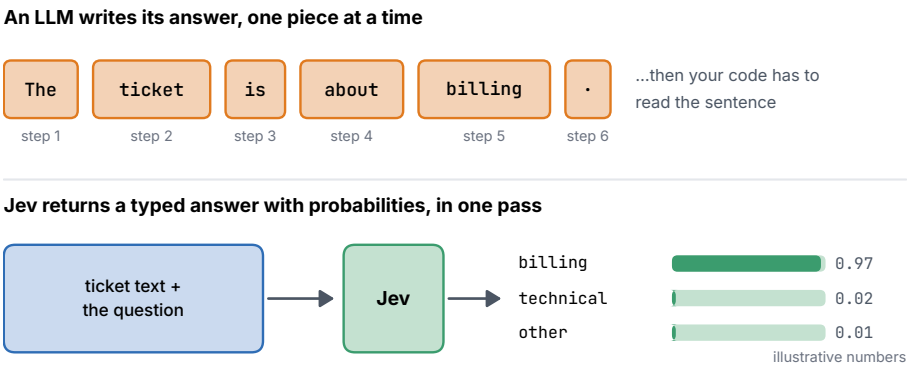


Figure 1.4. The same question, two ways. An LLM writes its answer piece by piece, and your code has to read the sentence. A decision model returns the answer as data, with a probability for every option, in one pass.

and a typed question, "pick one of these labels" or "yes or no?", and it returns the answer as data, with a probability for every option (Figure 1.4). TypeSafe calls it a *System One* model, after the psychologist Daniel Kahneman's name for fast, intuitive thinking (Kahneman 2011).

One honest note. I had no access to Jev's API while writing, so every Jev answer in this book comes from a stand-in I built that copies Jev's public interface. Those numbers are marked *synthetic*. Asked about one alert, an overnight flood of sign-in requests, the stand-in says 0.68: probably an attack. It wasn't. That isn't a failure. A 0.68 is a claim that *about two in three alerts like this* are attacks; this one was in the other third. Whether a model's 0.68 really means two in three is something we can test, and Chapters 3 and 11 do.

That's the whole map (Figure 1.5). Rules weren't enough, so machines learned from examples. Models grew, learned language, then learned to look things up and to act. And inside those loops, most of the work turned out to be *deciding*: quickly, cheaply, with probabilities that honestly say how sure they are.

The next three chapters stay at the bottom of that map on purpose, because everything above it rests on one idea: an answer should be a probability, and the probability should mean what it says.

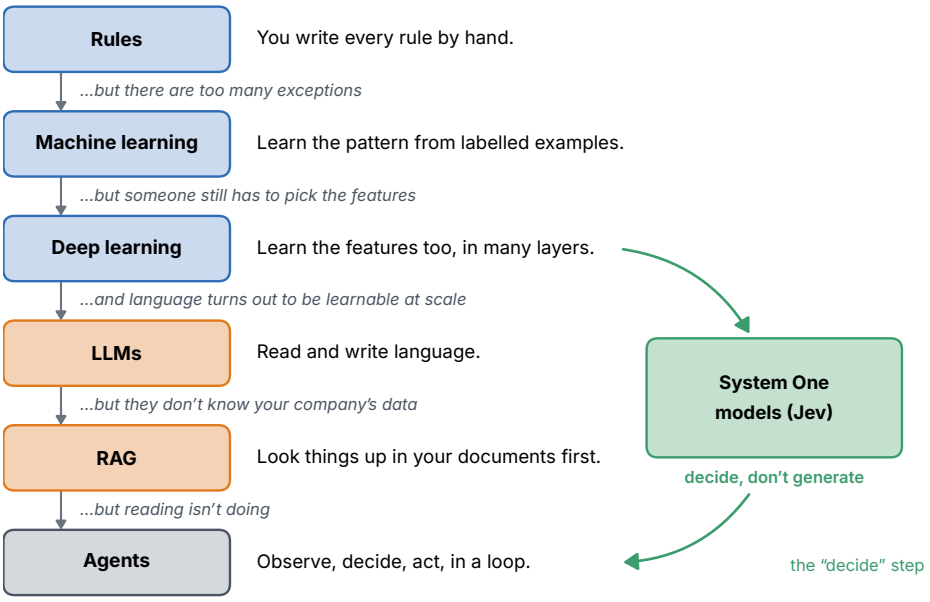


Figure 1.5. The story so far. Each step exists because the one before it hit a problem it couldn’t solve. System One models like Jev grow out of deep learning and fit into the “decide” step of an agent.

● KEEP THESE

- **Machine learning** is getting a computer to work out the rules itself, from examples where the answer is already known.
- A **model** is a function whose settings came from examples rather than from a programmer.
- A model learns exactly what you ask for: ask the wrong question, and it learns the wrong thing.
- A good model doesn’t just answer: it says how sure it is, and it’s right about how sure it is.

EXERCISES

1. Name one system you use every week that follows rules, and one that learned from examples. What gave each away?
2. Our best rule made 1,487 mistakes and caught almost nothing. Write one sentence for the question you *wish* we had asked the machine instead.

Next: “how likely” sounds like a small change. It’s the most important idea in the book, and it hides a trap that fools doctors, judges and security teams.

CHAPTER 2

Probability, and how a machine learns it

But to us, probability is the very guide of life.

— Joseph Butler, The Analogy of Religion, 1736

Let me guess what went through your head when you saw the word "probability".

"Coins and dice. I did this at school, didn't enjoy it much, and I'm fairly sure I don't need it to build AI systems."

It's a reasonable thought. It's also the most costly mistake people make in this field.

In the last chapter, a machine learned what we asked and was still useless, because we asked for yes-or-no answers. The fix was to ask for a *chance* instead. This chapter does two things. First, it makes sure you can read a chance without being fooled. That includes the one trap that catches doctors, judges and security teams. Then it builds a model that learns chances from examples, in about twenty lines of code.

There's no difficult maths here. There are a few ideas that, once you understand them, you'll use for the rest of your career.

BY THE END OF THIS CHAPTER

- Say what "a 70% chance" means in a way you could check.
- Notice the base-rate trap before it costs you, and update a probability by multiplying odds.
- Read precision and recall, and explain why accuracy can lie.
- Build a model that adds up evidence into a probability, score it with log loss, and train it by rolling downhill.
- Recognise overfitting and leakage, the two ways a model fools you during training.

What “70%” means

Your weather app says there’s a 70% chance of rain tomorrow.

Tomorrow comes. It doesn’t rain.

Was the forecast wrong?

Not necessarily. One day can’t tell you. What *can* tell you is the forecaster’s record. Collect every day they said “70%”. If it rained on about 70 of every 100 of those days, the forecaster is doing their job. A dry day is just one of the 30.

So here’s the idea to remember:

> KEY IDEA
>
> **A probability is a claim about how often something happens among cases like this one.**

That definition is useful because you can *check* it. When a model tells you an alert is 70% likely to be an attack, it’s making the same kind of promise as the weather app. We can check it in the same way.

The two most important words: “among what?”

At Kestrel Logistics, about 7.6% of all alerts turn out to be real attacks. That’s the probability of an attack **among all alerts**. Among alerts with a threat-intel score of 0.7 or more, the share is much higher. It’s the same question, but the answer depends on *which group* you’re asking about.

Statisticians write the second kind with a vertical bar, read as “given”: P(attack | bad score) is the chance of an attack *given* a bad score. And here’s where people get confused. P(attack | bad score) and P(bad score | attack) look like the same thing reversed. They’re not the same.